INTERNSHIP ASSIGNMENT REPORT

AWS EC2 Static Website Deployment using Nginx & Docker

Submitted by

Priyamvada

B.Tech Information Technology

priyamvadachaudhary7078@gmail.com

AWS | EC2 | Ubuntu | Nginx | Docker | Linux

Live Website

Public IPv4 Address:

http://35.154.253.141

Note:

The website was successfully deployed on an AWS EC2 Ubuntu instance and was accessible at the time of assignment submission.

1. Introduction

1.1 What is Cloud Computing?

Cloud computing means using computing resources like servers, storage and networking over the internet instead of owning physical hardware. Instead of buying a physical server, I can rent a virtual server from a cloud provider and use it whenever I want. This is cheaper and much easier to set up for a small project like this one.

1.2 Why AWS?

AWS (Amazon Web Services) is one of the most widely used cloud platforms in the industry. I chose AWS for this project because it has a free tier which is good for learning, and because most companies use AWS or a similar cloud provider, so it made sense to get hands-on experience with it.

1.3 What is EC2?

EC2 stands for Elastic Compute Cloud. It is an AWS service that lets you create virtual machines (called instances) in the cloud. I used EC2 to create an Ubuntu server where I installed Nginx and Docker and hosted my website.

1.4 Why Nginx?

Nginx is a web server software. It is lightweight, fast, and easy to configure for hosting a static website. I used Nginx because it is one of the most commonly used web servers, and it was simple enough for me to install and configure as a beginner.

1.5 Why Docker?

Docker is a tool used to run applications inside containers. A container packages an application along with everything it needs to run, so it behaves the same way on any machine. I installed Docker as an additional (bonus) task to understand the basics of containerization, and tested it by running the Docker Hello World container.

1.6 Why Linux?

Most cloud servers, including the EC2 instance I used, run on Linux (Ubuntu in this case). Learning basic Linux commands is important for anyone working with cloud or DevOps tools, because almost everything on the server is done through the terminal.

2. Project Objectives

The main goal of this project was to deploy a static HTML website on an AWS EC2 server and understand the basic workflow of cloud deployment. The specific objectives are listed below.

- To launch and configure an AWS EC2 Ubuntu instance.
- To understand and configure Security Groups for controlling access to the server.
- To connect to the EC2 instance securely using SSH from Windows WSL.
- To update the Ubuntu server and install Nginx web server.
- To replace the default Nginx page with a custom HTML dashboard.
- To restart and verify the Nginx service after deploying the website.
- To host the website and verify that it is accessible from a browser using the Public IPv4 address.

- To verify the deployment locally using the curl command.
- To practice basic Linux commands used in day-to-day server administration.
- To install Docker on the same server and verify it by running the Docker Hello World container, as a bonus task.

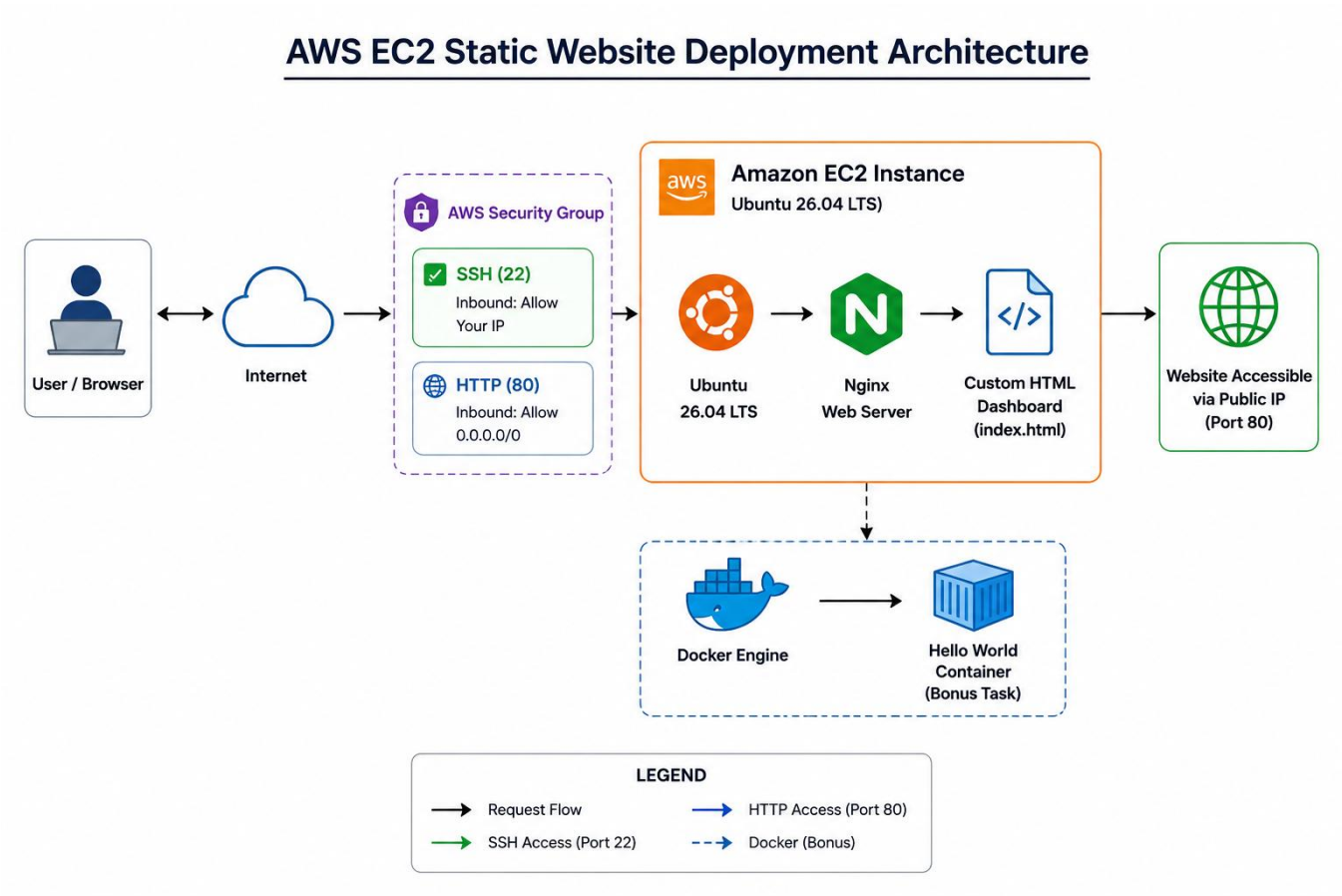
3. Tools and Technologies

The table below lists the tools and technologies used in this project, along with the reason for using each one.

Technology	Purpose	Why it was used
AWS EC2	To create a virtual server in the cloud	Free tier available, industry standard cloud platform
Ubuntu 26.04	Operating system for the server	Beginner friendly and widely used Linux distribution
Linux	Managing the server through terminal	Most cloud servers run on Linux
Nginx	Web server to host the website	Lightweight, fast and easy to configure
Docker	Running containerized applications	To learn basics of containerization
HTML5	Building the website content	Simple to create a static dashboard page
Git	Version control for project files	To track changes in the project
GitHub	Storing project files online	Easy to share and back up the project
Windows WSL	Terminal environment on my laptop	To run SSH and Linux commands from Windows

4. System Architecture

The diagram below shows the overall flow of the system, from a user's request to the final website being served.



4.1 Architecture Explanation

The flow of the system is straightforward and can be explained in the following steps:

- Internet: A user opens a browser and types the Public IPv4 address of the EC2 instance.
- Security Group: The request first passes through the Security Group, which acts like a firewall and only allows traffic on the ports I opened (HTTP and SSH).
- EC2 Ubuntu Server: Once allowed, the request reaches the EC2 Ubuntu server.
- Nginx: Nginx, which is running on the server, receives the request and restarts or reloads only when the configuration or website files change, then serves the website files.
- HTML Website: The custom HTML dashboard page that I created is returned to the user's browser.

Docker is installed on the same EC2 server, separately from the website hosting setup. It was used only to verify the installation by running the Docker Hello World container, and does not affect the website hosting flow described above.

5. Implementation

This section covers every step I performed to complete this project, from launching the EC2 instance to verifying the Docker installation. For each step I have mentioned the objective, the commands I used, an explanation, the output I got, and a screenshot.

Step 1: Launching EC2 Instance

Objective

To create a virtual server on AWS where I could install Nginx and host my website.

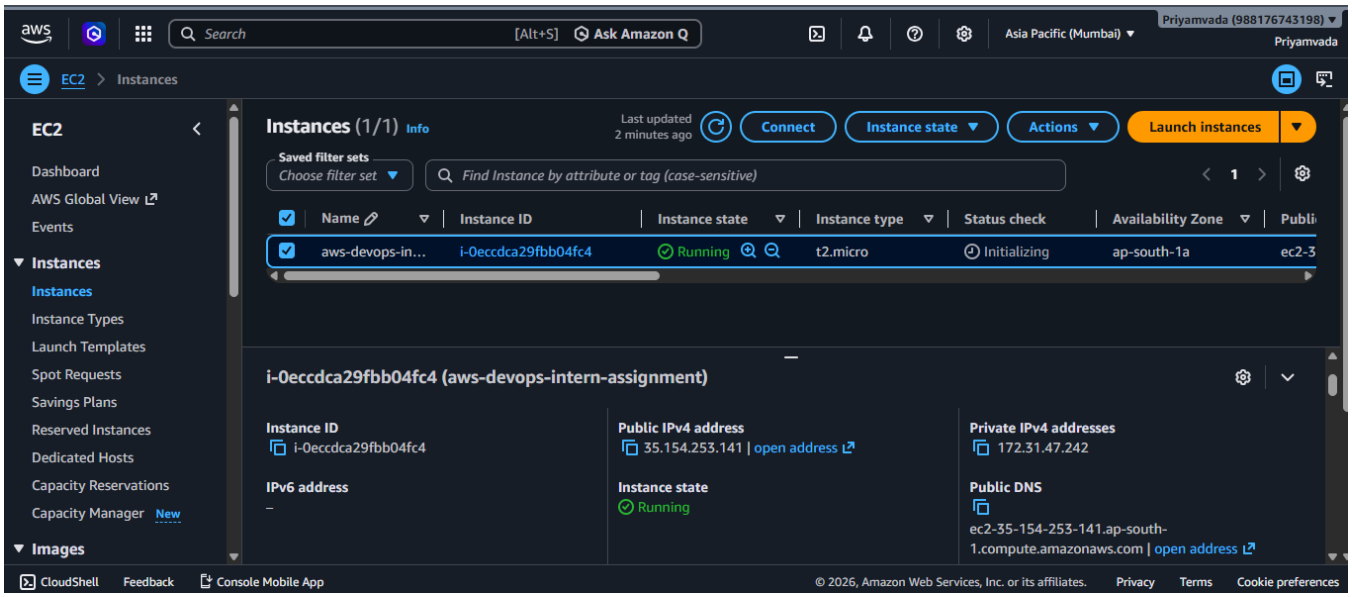
Explanation

I logged into the AWS Management Console and launched a new EC2 instance. I selected Ubuntu 26.04 as the operating system because it is a stable and beginner friendly Linux distribution, and most online resources for Nginx and Docker are written for Ubuntu. I chose the Mumbai (ap-south-1) region since it is closest to my location and gives lower latency. I selected a t2.micro instance type because it comes under the AWS Free Tier, which means I would not be charged for using it within the free tier limits. I also created a new key pair (.pem file) during launch, which I used later to connect to the server through SSH.

Output

The EC2 instance was successfully launched and its status changed to "Running" in the AWS console, with a Public IPv4 address assigned to it.

Screenshot



Step 2: Configuring Security Group

Objective

To control which type of traffic is allowed to reach the EC2 instance.

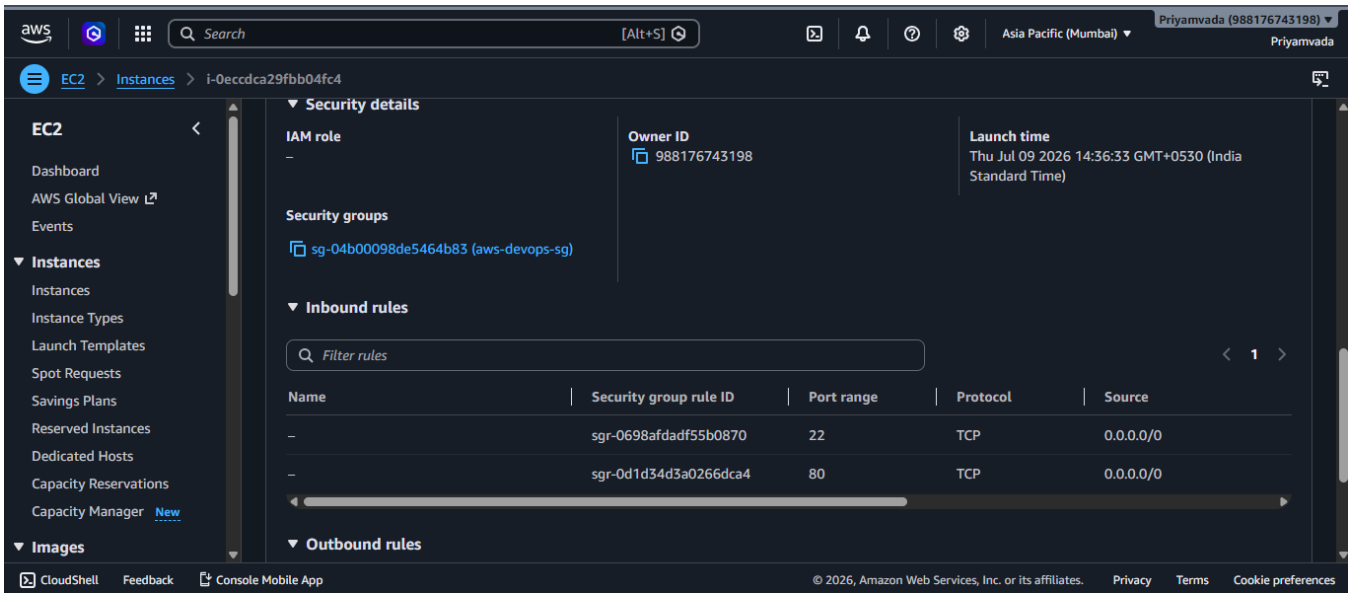
Explanation

By default, AWS creates a Security Group when launching an instance. I edited the inbound rules of this Security Group to allow SSH (port 22) so that I could connect to the server from my laptop, and HTTP (port 80) so that the website could be accessed from a browser. I restricted SSH access to my own IP address for better security, and kept HTTP open to all (0.0.0.0/0) since the website needed to be publicly accessible.

Output

The Security Group showed two inbound rules: one for SSH on port 22 and one for HTTP on port 80.

Screenshot



Step 3: Connecting via SSH from Windows WSL

Objective

To securely log into the EC2 Ubuntu server from my local machine.

Commands Used

```
chmod 400 my-key.pem
ssh -i my-key.pem ubuntu@<Public-IPv4-Address>
```

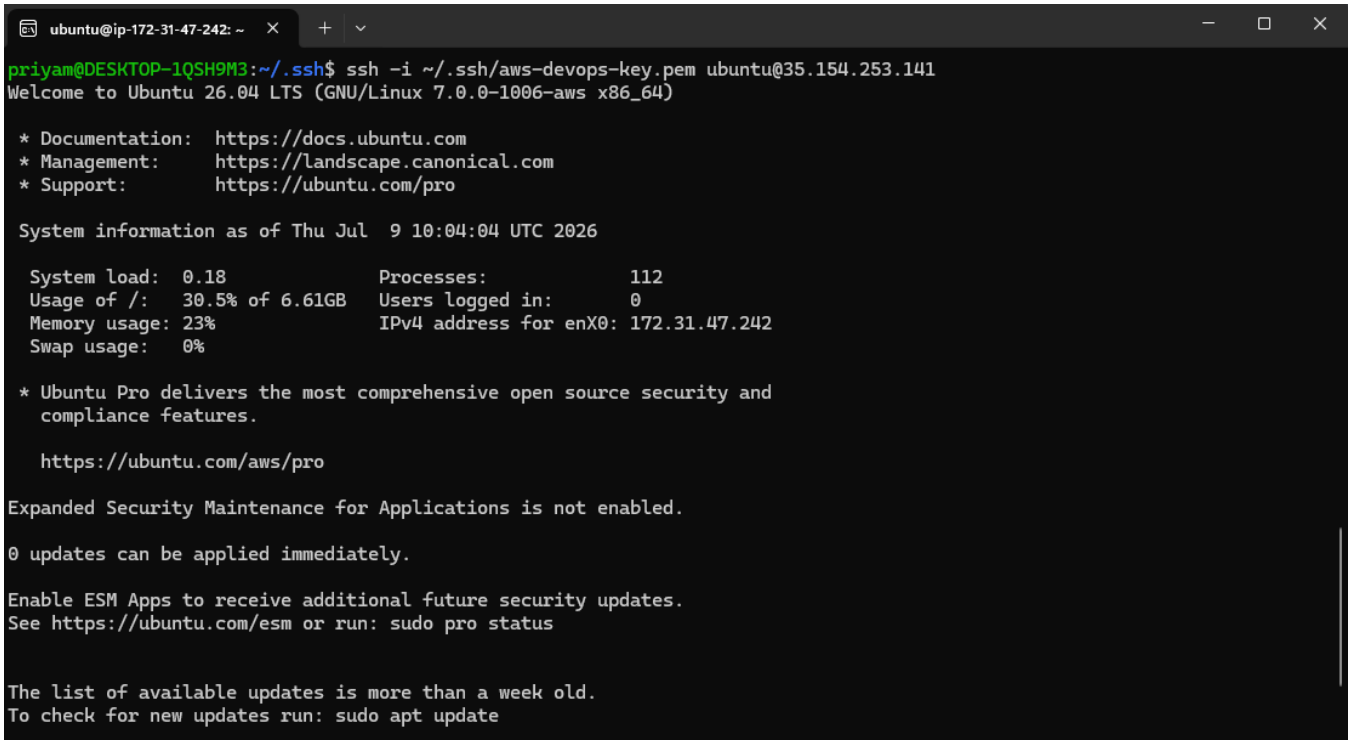
Explanation

I used Windows WSL (Windows Subsystem for Linux) as my terminal because it gives a proper Linux-like environment on Windows, which made running SSH and Linux commands much easier than using PowerShell. Before connecting, I had to change the permission of my .pem key file using chmod 400, otherwise SSH refused to use the key due to it being too open. After that, I connected to the server using the ssh command with my key file and the public IP address of the instance, using "ubuntu" as the default username for Ubuntu AMIs.

Output

I was successfully logged into the EC2 Ubuntu server, and the terminal prompt changed to show the Ubuntu server's hostname instead of my local machine.

Screenshot



Step 4: Installing Nginx

Objective

To install the web server software that would host my static website.

Commands Used

```
sudo apt install nginx -y
```

Explanation

I installed Nginx using the apt package manager. This command downloaded and installed Nginx along with its default configuration files. Once installed, Nginx automatically started running as a background service on the server.

Output

The terminal showed Nginx being downloaded, unpacked, and configured, and ended with a success message confirming the installation.

Screenshot

```
Fetched 30.4 MB in 3s (8588 kB/s)
58 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-47-242:~$ sudo apt install nginx -y
Building dependency tree... 50%
Installing:
  nginx

Installing dependencies:
  nginx-common

Suggested packages:
  fcgiwrap  nginx-doc  ssl-cert

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 58
  Download size: 664 kB
  Space needed: 1876 kB / 4654 MB available

Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 nginx-common all 1.28.3-2ubuntu1.6 [37.3 kB]
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 nginx amd64 1.28.3-2ubuntu1.6 [627 kB]
Fetched 664 kB in 0s (22.6 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 85303 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1.6_all.deb ...
Unpacking nginx-common (1.28.3-2ubuntu1.6) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1.6_amd64v3.deb ...
Unpacking nginx (1.28.3-2ubuntu1.6) ...
Setting up nginx-common (1.28.3-2ubuntu1.6) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/usr/lib/systemd/system/nginx.service'.
Setting up nginx (1.28.3-2ubuntu1.6) ...
 * Upgrading binary nginx                                                                                    [ OK ]
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for ufw (0.36.2-9build1) ...
```

Step 5: Verifying Nginx Service Status

Objective

To confirm that the Nginx service was actually running after installation.

Commands Used

```
sudo systemctl status nginx
```

Explanation

I used the systemctl status command to check whether Nginx was active and running. This is an important step because installing a service does not always mean it started correctly, so I wanted to confirm it myself before moving forward.

Output

The output showed the Nginx service as "active (running)", along with the process ID and recent log lines confirming that Nginx had started successfully.

Screenshot

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-47-242:~$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-07-09 10:16:06 UTC; 3min 38s ago
 Invocation: f547aa05d36748409c52350b10ab15f4
       Docs: man:nginx(8)
    Process: 1774 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 1776 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 1804 (nginx)
      Tasks: 2 (limit: 657)
     Memory: 2.3M (peak: 5.2M)
        CPU: 33ms
     CGroup: /system.slice/nginx.service
             ├─1804 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─1807 "nginx: worker process"

Jul 09 10:16:06 ip-172-31-47-242 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jul 09 10:16:06 ip-172-31-47-242 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-47-242:~$
```

Step 6: Replacing the Default Nginx Page

Objective

To host my own custom HTML dashboard instead of the default Nginx welcome page.

Commands Used

```
cd /var/www/html
sudo rm index.nginx-debian.html
sudo nano index.html
```

Explanation

By default, Nginx serves a welcome page located at /var/www/html/index.nginx-debian.html. I removed this default file and created a new index.html file in the same location using the nano text editor. I wrote a simple, responsive HTML dashboard page with basic CSS styling directly on the server, and saved the file.

Output

The default Nginx welcome page was replaced, and the new index.html file was saved in the /var/www/html directory

Step 7: Restarting the Nginx Service

Objective

After replacing the default Nginx page with my custom index.html file, I restarted the Nginx service to make sure the latest website files were loaded correctly by the server.

Commands Used

```
sudo systemctl restart nginx
sudo systemctl status nginx
```

Explanation

I first ran systemctl restart nginx so that Nginx would reload and pick up the new index.html file. Right after that, I ran systemctl status nginx again to double check that the service had restarted properly and was not showing any errors.

Even though just replacing an HTML file does not always require restarting Nginx, since Nginx serves static files directly without needing a reload, I still restarted the service as a good deployment habit. It removes any doubt about caching or the old file still being served, and it is a quick way to confirm that the web server is healthy right after making a change.

Output

The status command showed "Active (running)" again, confirming that the web server restarted cleanly and was working correctly after the update.

Screenshot

```
*** System restart required ***
Last login: Thu Jul  9 16:30:06 2026 from 157.119.83.208
ubuntu@ip-172-31-47-242:~$ sudo systemctl restart nginx
ubuntu@ip-172-31-47-242:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-07-10 09:14:09 UTC; 14s ago
 Invocation: 1a36bb07a44146a893daadb9156ece4e
       Docs: man:nginx(8)
    Process: 20477 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 20479 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 20480 (nginx)
      Tasks: 2 (limit: 657)
     Memory: 2.5M (peak: 2.8M)
        CPU: 23ms
     CGroup: /system.slice/nginx.service
             ├─20480 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─20481 "nginx: worker process"

Jul 10 09:14:08 ip-172-31-47-242 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jul 10 09:14:09 ip-172-31-47-242 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
```

Figure: Restarting and Verifying the Nginx Service

Step 8: Verifying Website in Browser

Objective

To check that the website was accessible publicly through a web browser.

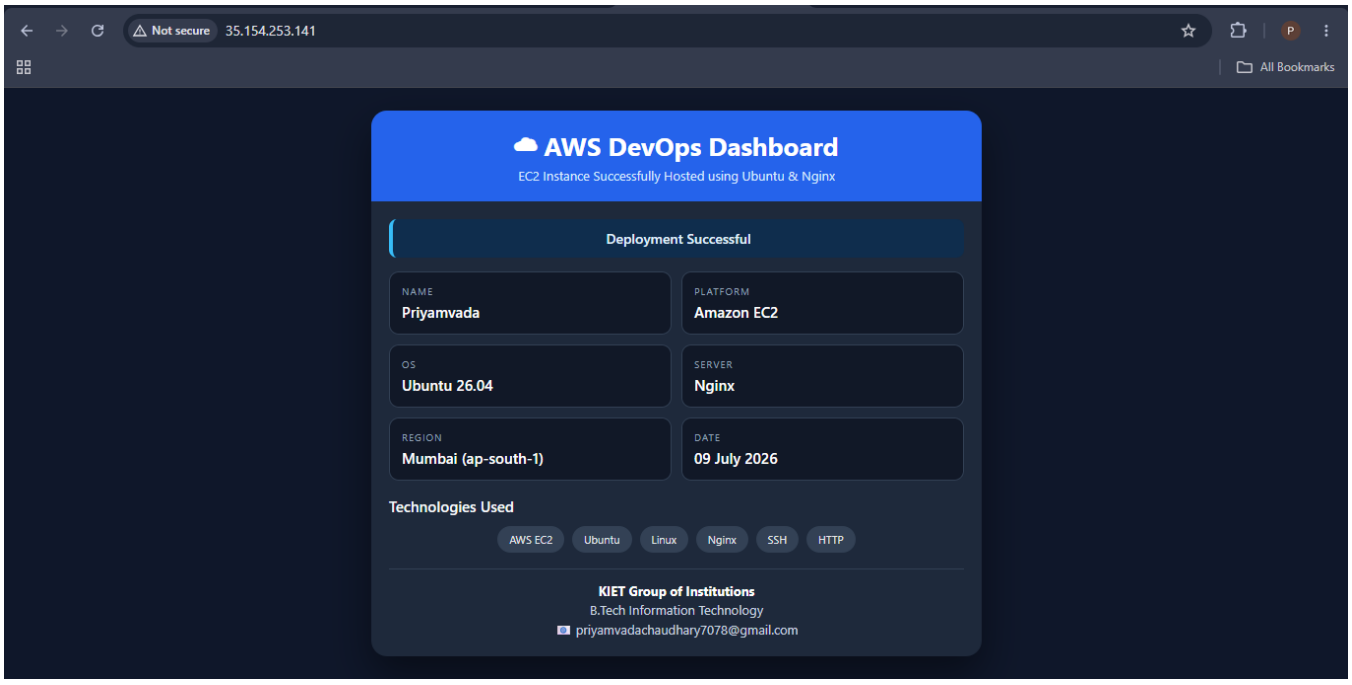
Explanation

I opened a browser on my laptop and entered the Public IPv4 address of the EC2 instance. Since Nginx listens on port 80 by default, I did not need to specify any port number in the URL.

Output

The browser displayed my custom HTML dashboard page instead of the default Nginx page, confirming that the website was successfully hosted and publicly accessible.

Screenshot



Step 9: Verifying Deployment using curl localhost

Objective

To confirm from within the server itself that Nginx was correctly serving the website content.

Commands Used

```
curl localhost
```

Explanation

In addition to checking the website in a browser, I also ran the curl command directly on the server. This command sends an HTTP request to the local Nginx server and prints the raw HTML response. This helped me confirm that the issue (if any) was not related to networking or the Security Group, but purely about whether Nginx was serving the correct file.

Output

The terminal printed the full HTML content of my custom dashboard page, confirming that Nginx was serving the correct file locally.

Screenshot

```
ubuntu@ip-172-31-47-242:~$ curl http://localhost
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>AWS DevOps Assignment</title>

<style>

*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:'Segoe UI',sans-serif;
}

body{
background:#0f172a;
display:flex;
justify-content:center;
align-items:center;
height:100vh;
overflow:hidden;
padding:12px;
}

.container{
width:620px;
background:#1e293b;
border-radius:16px;
overflow:hidden;
box-shadow:0 18px 40px rgba(0,0,0,.45);
}
```

Step 10: Basic Linux Commands

Objective

To practice and note down commonly used Linux commands while working on the server.

Commands Used

```
hostname
pwd
whoami
```

```
free -h
df -h
```

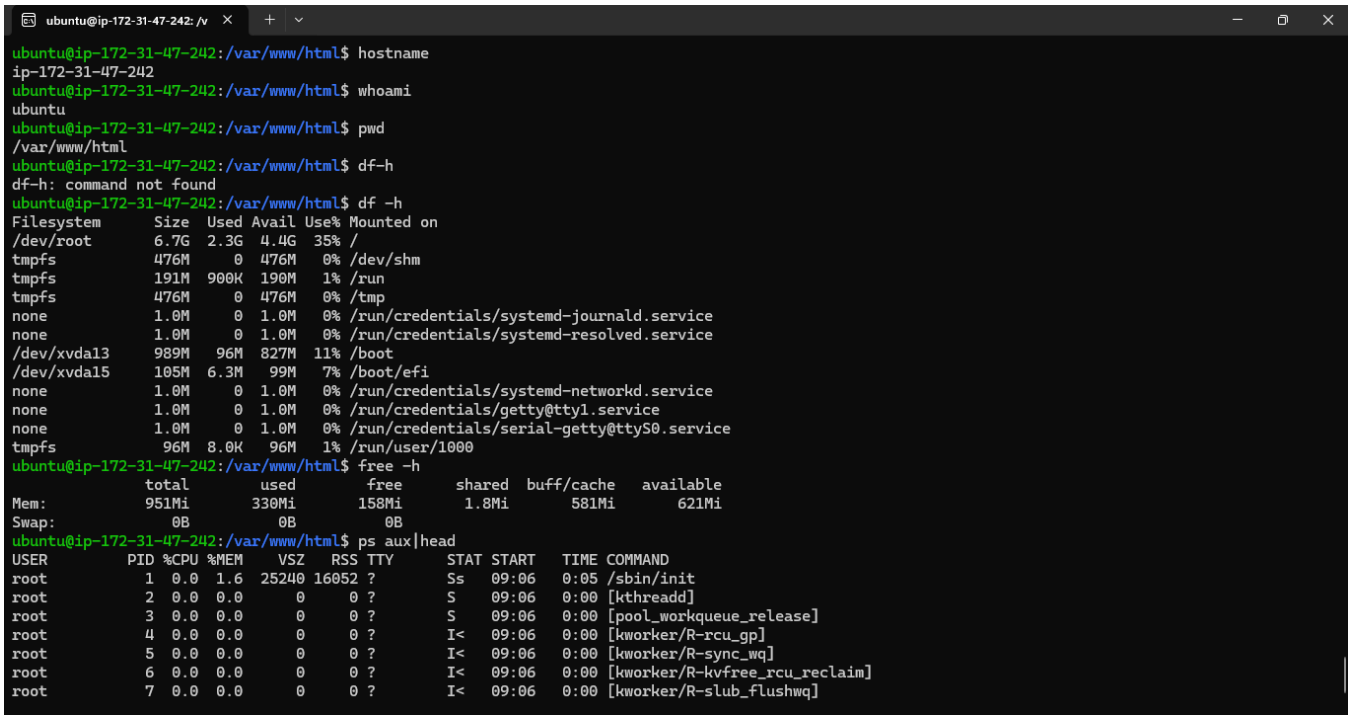
Explanation

While working on the server, I used a few basic Linux commands to check system information. hostname showed the server's name, pwd showed my current working directory, whoami showed the logged in user, free -h showed memory usage in a readable format, and df -h showed disk space usage in a readable format. These commands are simple but useful for understanding the basic state of a server.

Output

Each command printed the relevant system information in the terminal, such as the server hostname, current directory path, logged in username, available memory, and available disk space.

Screenshot



6. Docker Bonus Task

Installing Docker was not part of the core task of hosting a website, but I did it as a bonus task to get some basic hands-on exposure to containerization, since it is commonly used alongside tools like Nginx in real DevOps workflows.

6.1 Why Docker Was Installed

I wanted to understand what a container is and how it is different from a normal application. Docker is one of the most widely used tools for this, so I installed it on the same EC2 server to try it out after finishing the main website hosting task.

6.2 Installation Command

```
sudo apt install docker.io -y
```

6.3 Starting the Docker Service

```
sudo systemctl start docker
```

6.4 Enabling Docker on Boot

```
sudo systemctl enable docker
```

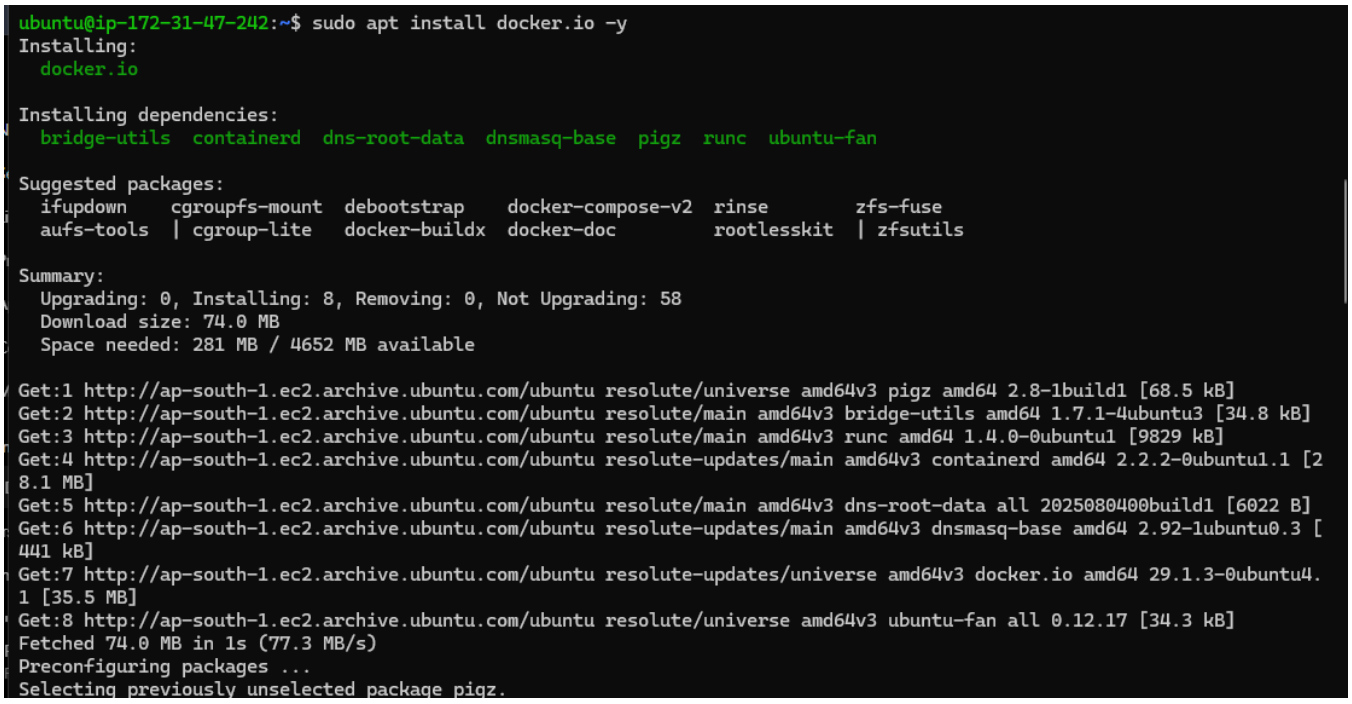
6.5 Checking Docker Version

```
docker --version
```

6.6 Docker Hello World Output

```
sudo docker run hello-world
```

Running this command downloaded the hello-world image from Docker Hub and ran it as a container. The output confirmed that my Docker installation was working correctly.



,

```
ubuntu@ip-172-31-47-242:~$ sudo systemctl start docker
ubuntu@ip-172-31-47-242:~$ sudo systemctl enable docker
ubuntu@ip-172-31-47-242:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
ubuntu@ip-172-31-47-242:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:96498ffd522e70807ab6384a5c0485a79b9c7c08ca79ba08623edcad1054e62d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/
```

7. Linux Commands Used

Below is a summary table of the Linux commands I used during this project, along with their purpose and a short description of the output.

Command	Purpose	Output Description
sudo apt update	Refresh package index	List of updated package sources
sudo apt upgrade -y	Upgrade installed packages	Summary of upgraded packages
sudo apt install nginx -y	Install Nginx web server	Nginx installed successfully
sudo systemctl status nginx	Check Nginx service status	Shows active/inactive state
sudo systemctl restart nginx	Restart Nginx service	Reloads Nginx with the latest files
cd /var/www/html	Change to website directory	No output, changes directory
sudo nano index.html	Edit website HTML file	Opens file in text editor
curl localhost	Test website from the server	Prints raw HTML of the page
hostname	Show server hostname	Prints server name
pwd	Show current directory	Prints full directory path
whoami	Show current logged in user	Prints username (ubuntu)
free -h	Show memory usage	Prints RAM usage in readable form
df -h	Show disk usage	Prints disk space in readable form
sudo apt install docker.io -y	Install Docker	Docker installed successfully
sudo systemctl start docker	Start Docker service	No output if successful

sudo systemctl enable docker	Enable Docker on boot	Creates symlink confirmation
docker --version	Check Docker version	Prints installed Docker version
sudo docker run hello-world	Test Docker installation	Prints hello-world confirmation message

8. Challenges Faced

Like any hands-on project, I faced a few issues while completing this assignment. Below are the main challenges and how I resolved them.

8.1 SSH Private Key Permission Issue in WSL

When I first tried to connect to my EC2 instance using SSH from WSL, I got a permission error saying my private key file was too open. I faced this because the .pem file had default permissions that were not secure enough for SSH. I solved this by running chmod 400 on the key file, which restricted its permissions, after which SSH allowed me to connect.

8.2 Understanding Security Groups

Initially, I was not sure why I could not access my website even though Nginx was running correctly. I later understood that this was because I had not opened port 80 in the Security Group. Once I added an inbound rule allowing HTTP traffic on port 80, the website became accessible from the browser.

8.3 Replacing the Default Nginx Page

When I tried to edit the default index file directly, I faced a permission denied error because the file belonged to the root user. I resolved this by using sudo while editing and removing the file, which gave me the required permissions.

8.4 Making the Website Accessible

Even after installing Nginx and updating the HTML file, the website did not load in the browser at first. After checking again, I realized this was actually related to the Security Group issue mentioned above, and once that was fixed, the website loaded properly.

8.5 Installing Docker

While installing Docker, I was not sure whether to use the docker.io package from apt or Docker's official installation script. Since this was a bonus task and I wanted to keep things simple, I went with the docker.io package available directly through apt, which installed without any issues.

8.6 Verifying Services

At first, I assumed that installing Nginx and Docker was enough for them to work correctly. I later learned that it is important to actually check the service status using systemctl status, and restart the service if needed, rather than just assuming it is running after installation or after a file change.

9. Learning Outcomes

This project gave me a lot of practical exposure to cloud computing and Linux, which I had mostly read about in theory before this. Some of the key things I learned are listed below.

- I learned how to launch EC2 instances and choose the right settings like region and instance type.
- I understood how SSH works and why key based authentication is used instead of a simple password.
- I gained confidence working with Linux and using the terminal for everyday server tasks.
- I learned how Nginx hosts websites, how it serves files from a specific directory, and why restarting the service after a change is a good habit.
- I understood the basics of Docker, including images, containers, and how to run a simple container.
- I became comfortable using terminal commands like cd, pwd, whoami, free -h and df -h.
- I understood the importance of Security Groups and how they control access to a server.
- I learned how to debug simple issues by checking service status instead of assuming everything works after installation.

10. Future Improvements

This project covered the basic deployment of a static website along with a Docker bonus task. There are several areas I would like to explore and add in the future, once I learn more about them. These have not been implemented in the current project.

- Adding HTTPS to the website using an SSL certificate for secure access.
- Using Route 53 for a custom domain name instead of accessing the site through an IP address.
- Setting up CloudWatch to monitor server performance and logs.
- Using CloudFormation to automate the creation of AWS resources.
- Using GitHub Actions for automating deployment whenever code changes are pushed.
- Setting up a basic CI/CD pipeline for the website.
- Using Docker Compose to manage multiple containers together, once I work on a multi-container project.

11. Conclusion

Overall, this project helped me move from just reading about cloud computing to actually doing it myself. I launched an EC2 instance, connected to it using SSH, installed and configured Nginx, hosted a custom website, restarted and verified the Nginx service after deployment, and checked everything both through a browser and using curl. I also installed Docker as a bonus task and confirmed it was working using the Hello World container.

I faced a few real issues along the way, like the SSH key permission error and the Security Group configuration, but working through them helped me understand these concepts much better than just reading about them would have.

This project gave me practical, hands-on learning in several areas, including AWS EC2 provisioning, basic Linux server management, SSH connectivity, Nginx deployment and service management, static website hosting, Docker basics, and documenting a project properly on GitHub.

This project strengthened my understanding of AWS and Linux, and gave me a good starting point to explore more advanced DevOps tools and concepts in the future.